

IIC2440 – Procesamiento de Datos Masivos

I2

Instrucciones. La interrogación tiene una duración de 120 minutos.

Resuelve cada problema en una hoja separada. Las preguntas cortas generalmente tienen una mayor probabilidad de estar correctas. Todas las preguntas tienen 2 puntos; la nota final se calcula como la suma de los puntos obtenidos en cada pregunta, más un punto base.

Las preguntas se hacen en voz alta; solo en el intervalo de tiempo **de 18:30 a 18:50**.

Problema 1: Recomendación de Conexiones en Redes Sociales

Disponemos de un conjunto de datos que representa una red social dirigida. La relación fundamental es el vínculo **sigue a** ($A \rightarrow B$). El objetivo es implementar un sistema de sugerencias basado en la **transitividad de las relaciones**: si A sigue a B , y B sigue a C , entonces C es un candidato potencial para ser seguido por A .

El **peso** de la sugerencia para el par (A, C) se define como la cantidad de intermediarios B que conectan a A con C .

Restricciones Técnicas: Debe resolver este problema utilizando estrictamente el modelo de programación *MapReduce*. Cuenta con una operación especial de JOIN que permite cruzar dos conjuntos de datos por una clave común dentro de un mismo flujo.

Input: Un iterable de pares (`User_ID`, `Followed_ID`).

1. **[1.2 pto] Implementación:** Diseñe el algoritmo MapReduce para obtener, para cada usuario, una lista de candidatos a seguir junto con su peso. Detalle las funciones **Map** y **Reduce**.
2. **[0.8 pts] Análisis de Escalabilidad:** Suponga que en la red existe un nodo “Celebridad” (por ejemplo, una cuenta oficial) que sigue y es seguido por el 80% de la plataforma.
 - ¿Qué impacto tendría este nodo en el rendimiento de la fase de *Shuffle*?
 - Explique el fenómeno de **Data Skew** (sesgo de datos) y cómo afectaría específicamente a la memoria de un *Reducer* encargado de procesar a dicha “Celebridad”.

Problema 2: Propagación de Confianza

Se modela una red de espionaje como un grafo dirigido donde una arista (i, j) indica que el agente i entrega información al agente j . Los agentes se clasifican en: **Confiables** (S), **Hostiles** (P) y **Desconocidos** (Q).

Se desea asignar a cada agente un puntaje de confiabilidad $s_i \in [-1, 1]$, donde > 0 es confiable y < 0 es hostil.

Modelo Matemático

Se define el vector de estado inicial y^0 con cada i representando a un agente:

$$y_i^0 = \begin{cases} +1 & \text{si } i \in S \\ -1 & \text{si } i \in P \\ 0 & \text{si } i \in Q \end{cases}$$

El puntaje se actualiza de forma iterativa mediante la regla:

$$s^{(t)}(i) = (1 - \alpha) y_i^0 + \alpha \sum_{j \in In(i)} \frac{s^{(t-1)}(j)}{Out(j)}$$

donde $\alpha \in (0, 1)$ es el factor de amortiguación, $In(i)$ es el conjunto de predecesores de i y $Out(j)$ es el grado de salida de j . La iteración inicia en $s^{(0)} = y^0$.

1. **[0.5 pt] Cálculo e interpretación.** Considere el sub-grafo con nodos $\{0, 1, 2\}$, donde $0 \in S$, $2 \in P$, $1 \in Q$, y aristas

$$\{(0, 1), (0, 2), (2, 1), (1, 0)\}.$$

Con $\alpha = 0,5$, calcule $s^{(1)}(0)$, $s^{(1)}(1)$ y $s^{(1)}(2)$. Interprete brevemente el signo y la magnitud de $s^{(1)}(1)$ a la luz de la red: ¿por qué ese valor para un agente desconocido que recibe información tanto del confiable como del hostil?

2. **[0.5 pt] Análisis de α .** ¿Qué ocurre cuando α tiende a 0 y cuando tiende a 1? Explique el rol que cumple en el sistema y de un ejemplo de grafo simple en el que la elección de un mal α (es decir, un α cercano a 0 o bien cercano a 1) no permita la estabilidad del sistema (oscilación).
3. **[0.5 pt] Sumideros** El modelo actual tiene sumideros, nodos que reciben información/peso, pero no lo distribuyen a nadie y se pierde. ¿Cómo mitigaba esto PageRank? ¿Podríamos hacer algo nosotros aquí?
4. **[0.5 pt] Implementación distribuida.**
 - Justifique por qué una iteración de este algoritmo es naturalmente paralelizable: ¿qué dato necesita el nodo i para calcular $s^{(t)}(i)$ y por qué esto permite distribuir el cómputo entre nodos i distintos?
 - Explique por qué **Apache Spark** es más eficiente que **MapReduce** para este algoritmo específico.

Problema 3: Apriori sobre streams

Una plataforma de e-commerce necesita tu ayuda! Cada una de las transacciones que hacen sus usuarios llegan como una canasta C_t (un conjunto de productos comprados juntos), donde $t = 1, 2, 3, \dots$ es el orden de llegada. El catálogo tiene $|E| = 10^6$ productos distintos y la plataforma recibe hasta 10.000 transacciones por segundo. Quieres correr Apriori sobre este stream para encontrar reglas de asociación con soporte $\geq s$ (no te interesa la confianza). El problema:

- No puedes guardar el stream completo en disco, son petabytes acumulados al mes.
 - Apriori clásico requiere múltiples pasadas sobre el dataset, pero en un stream cada canasta se ve una sola vez.
 - Tu memoria de trabajo permite mantener a lo más $K = 10^7$ canastas simultáneamente.
1. **[1 pto]** Para un tiempo arbitrario T , entrega un pseudocódigo que corra Apriori sobre un sample de tamaño K que esté distribuido uniformemente sobre las T canastas que han llegado hasta el tiempo T . Tu respuesta debe describir con precisión el muestreo, justificar por que el muestreo resulta uniforme, especificar bien los parámetros de Apriori.
 2. **[0.2 pto]** Explica qué errores puede tener tu estrategia (en términos de las reglas que descubres) y por qué.
 3. **[0.8 pto]** Sea $\varepsilon > 0$ un parámetro, supón ahora lo siguiente:

Después del tiempo T en que muestreamos un sample y corremos Apriori sobre él, el stream sigue llegando indefinidamente y la distribución subyacente de canastas no cambia: las canastas $C_{T+1}, C_{T+2}, \dots$ provienen de la misma distribución que generó $C_1, \dots, C_T$.

Con este supuesto, diseña un algoritmo Apriori sobre el stream de datos tal que la probabilidad de que entregues una regla I cuyo soporte observado sea mayor que $p_I + \varepsilon$, con p_I el soporte real de la regla I , sea menor que 0,0001.

Ayuda. Sea I una regla. Después del tiempo T podemos contar en cuántas de las m canastas posteriores aparecen todos los elementos de I , y calcular el soporte observado c_I/m , donde c_I es la cantidad de canastas en $C_{T+1}, \dots, C_{T+m}$ que tienen todos los elementos de I . Puedes usar la siguiente propiedad (Hoeffding): para cada $\varepsilon > 0$ se puede demostrar que existe un N tal que para todo $m \geq N$ y para toda regla I con elementos de E , se tiene que:

$$P(|c_I/m - p_I| \geq \varepsilon) \leq 2e^{-2m\varepsilon^2}$$

donde p_I es el soporte real de I .